

LABORATORIO 05

¿Y si nadie los borró?

Operación de tópicos y retención.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 60 minutos, y varios son de espera

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber **visto desaparecer 100 mensajes de un tópico sin que nadie ejecute un comando de borrado**. Y vas a saber exactamente por qué desaparecieron.

Este es el laboratorio que explica **por qué Kafka no es una base de datos**. En una base de datos, lo que insertaste está hasta que alguien lo borre. En Kafka pasa lo contrario. Lo que escribiste se va solo, y la única razón por la que sigue ahí es que el plazo que tú configuraste todavía no venció.

LO QUE ESTE LABORATORIO DEMUESTRA

Kafka borra mensajes solo, cuando se lo pediste, y no te avisa.

Son tres partes y las tres se ven en pantalla. **Solo**, porque nadie va a ejecutar un comando de borrado en toda la guía. **Cuando se lo pediste**, porque el plazo lo vas a escribir tú en el paso 2. **Y no te avisa**, porque no hay log, ni alerta, ni métrica. El único rastro va a ser que un número cambió.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo	Que el ícono de la ballena esté activo
6 GB de RAM asignados a Docker	Ajustes de Docker Desktop, pestaña <i>Resources</i> . Con menos, los tres servidores no arrancan
Git Bash abierto	Es la terminal que usamos en todo el curso
El repositorio del curso	Debe estar en <code>C:\KAFKA\curso-kafka-sunat</code>
Los puertos 9092, 9093, 9094 y 8090 libres	El paso 1 te avisa si no lo están

ANTES DE TOCAR NADA · DOS COSAS

Uno. Actualiza el repositorio, porque el material se corrige durante el curso:

```
cd /c/KAFKA/curso-kafka-sunat
git pull
```

Dos. Comprueba si tienes otro laboratorio del curso levantado:

```
docker ps --filter name=kafka --format "{{.Names}}\t{{.Label}}\n\"com.docker.compose.project\"}"
```

La segunda columna te dice de qué laboratorio es cada contenedor. **Si aparece alguno, bájalo tú**, poniendo el número que veas:

```
docker compose -p novatech-labNN down -v --remove-orphans
```

Y NO CONFÍES EN QUE EL SCRIPT LO HAGA POR TI

`bin/start-lab.sh` elimina los contenedores llamados `kafka-broker-1`, `-2`, `-3`, `kafbat-ui` y `gps-producer` que sean de otro laboratorio del curso. **No elimina `kafka-broker`, que es el del laboratorio 01 y no lleva número, ni `kafka-broker-4`, que es el del laboratorio 08.** Si vienes de uno de esos dos, el puerto va a seguir ocupado y el arranque va a fallar.

El caso

Un lunes por la mañana el área de fiscalización de **NovaTech Logistics** abre el tablero y la serie de comprobantes del jueves anterior no está.

No hay un `DELETE` en ningún log de auditoría. Nadie tocó nada. El equipo de plataforma jura que el clúster estuvo arriba todo el fin de semana, y tiene razón. Estuvo arriba.

La sala se va a pasar la mañana en la pregunta obvia, «**¿quién los borró?**». Esa pregunta se puede investigar durante días sin llegar a ninguna parte, porque parte de un supuesto que nadie puso sobre la mesa. El supuesto de que alguien los borró.

Guarda la otra pregunta, que es la que vas a contestar hoy. **¿Y si nadie los borró?**

LA METÁFORA DE TODO EL CURSO · EL RESTAURANTE

Kafka es **un restaurante**. El **mozo** que toma y entrega los pedidos es el **broker**. Un **tipo de comanda**, como cocina fría, cocina caliente o barra, es el **tópico**. Los **sectores del salón** son las **particiones**. Las **libretas de respaldo** que copian cada comanda son las **réplicas**.

Hoy se suma una pieza nueva, y es la protagonista. El **espiche**, que es el pincho donde el mozo va clavando las comandas del turno.

Levantar el clúster de tres servidores

Qué vamos a hacer

Este laboratorio necesita tres servidores de Kafka trabajando juntos, más una interfaz web para mirarlos. Un script del curso los levanta todos.

A diferencia del laboratorio 01, aquí no vas a armar el archivo de configuración. Ya está resuelto, porque lo que se estudia hoy no es cómo se enciende un clúster. Es qué le pasa a los datos que hay dentro.

Los comandos

```
cd Capitulo_3/lab-05-operacion-topicos
chmod +x bin/*.sh kafka-cli/*.sh infra/scripts/*.sh
bin/start-lab.sh
```

cd Capitulo_3/lab-05-operacion-topicos — entra a la carpeta del laboratorio. **Todos los comandos de esta guía se ejecutan desde aquí**, no desde otra carpeta

chmod +x — le da permiso de ejecución a los scripts. En Windows con Git Bash a veces se pierde al clonar. **Si ya lo hiciste antes, ejecutarlo de nuevo no hace daño**

bin/*.sh kafka-cli/*.sh infra/scripts/*.sh — las tres carpetas que traen scripts. El asterisco significa «todos los archivos que terminan en `.sh`»

bin/start-lab.sh — el script que levanta el laboratorio. Crea los tres brokers, espera a que estén sanos, y levanta la interfaz web. **Antes limpia cinco nombres de contenedor que puedan haber quedado de otro lab del curso**, y esos cinco no son todos, como dice la caja de arriba

LO QUE VAS A VER · RECORTADO, EL ORIGINAL TRAE UN BANNER GRANDE

```
[1/4] Levantando contenedores del clúster NovaTech...
✓ Network novatech-lab05_novatech-network Created
✓ Container kafka-broker-2 Healthy
✓ Container kafka-broker-3 Healthy
✓ Container kafka-broker-1 Healthy
✓ Container kafbat-ui Started
✓ Container gps-producer Started
```

```
[2/4] Esperando a que los brokers estén operativos...
✓ 3/3 brokers operativos
```

```
[3/4] Esperando a que Kafbat UI esté disponible...
✓ Kafbat UI disponible
```

```
[4/4] Verificando productor GPS...
✓ Productor GPS activo
```

CLÚSTER NOVATECH OPERATIVO

Componentes:

Broker 1:	localhost:9092
Broker 2:	localhost:9093
Broker 3:	localhost:9094
Kafbat UI:	http://localhost:8090

VAS BIEN SI...

Leíste la línea `✓ 3/3 brokers operativos` y el bloque **CLÚSTER NOVATECH OPERATIVO** al final.

Compruébalo tú mismo con `docker ps --filter name=kafka-broker`. Tienen que salir **tres líneas**, con `kafka-broker-1`, `-2` y `-3`, y las tres con `Up` y `(healthy)` en la columna `STATUS`.

SI ALGO FALLA EN ESTE PASO

Dice `[ERROR] Docker no está corriendo` . Abre Docker Desktop y espera a que la ballena deje de moverse. Después repite el comando.

Dice `[ADVERTENCIA] Docker tiene 4 GB de RAM asignados` . Ve a los ajustes de Docker Desktop, pestaña *Resources*, y súbelo a 6 GB. Docker se reinicia solo. Con menos memoria el tercer broker se queda a medias.

Dice `port is already allocated` . Algo en tu máquina ya está usando el 9092, el 9093, el 9094 o el 8090. Averigua qué con `docker ps` y bájalo. Si no es de Docker, es un programa tuyo.

Dice `el nombre 'kafka-broker-1' lo tiene un contenedor que NO es del curso` . El script se niega a tocar nada ajeno, y hace bien. Resuélvelo tú con `docker rm -f kafka-broker-1` , pero **solo si sabes que ese contenedor es descartable**.

Dice `Timeout: solo 2/3 brokers están operativos` . Casi siempre es memoria. Sube Docker a 6 GB y ejecuta `bin/start-lab.sh` otra vez.

Fabricar el tópico que sí podemos ver morir

Qué vamos a hacer

Un tópico normal guarda sus datos siete días. Nadie va a esperar siete días. Vamos a crear uno con los plazos en segundos, para poder ver en una sola sesión lo que en producción tarda una semana.

CONCEPTO · TÓPICO

Un **tópico** es el nombre bajo el que se agrupan mensajes del mismo tipo. No se parece a una tabla ni a una cola. Es un registro que solo crece por el final, y leer de él no borra nada. En el restaurante es el **tipo de comanda**.

CONCEPTO · PARTICIÓN

Cada uno de los **pedazos en que se corta un tópico**. Un tópico de 6 particiones son 6 registros independientes, cada uno con su propio orden interno. Son los **sectores del salón**, y existen para que el trabajo se pueda repartir entre varios.

CONCEPTO · SEGMENTO

El **archivo físico** en el disco del broker donde se van escribiendo los mensajes de **una partición**, uno detrás de otro. Cada partición tiene en todo momento **un segmento activo**, que es donde se escribe, y cero o más **segmentos cerrados**.

EL ESPICHE · LA PIEZA DE HOY

El segmento es **el espiche**, ese pincho donde el mozo clava las comandas del turno.

Cuando el turno termina, el mozo **no va sacando comanda por comanda del pincho**. Baja el espiche entero al depósito, pone uno nuevo, y sigue. Y cuando llega la orden de limpiar lo viejo, **lo que se bota es el espiche completo**, no las comandas de a una.

De ahí sale la única regla que necesitas para entender todo lo demás. 🗑️ **Kafka no borra mensajes. Bota espiches enteros.**

El **plazo que Kafka respeta antes de botar datos viejos** de un tópico. Se configura en milisegundos, con `retention.ms`. Es un plazo por tópico, así que dos tópicos del mismo clúster pueden tener plazos completamente distintos.

Los dos parámetros, y por qué van juntos

`segment.ms` dice cada cuánto se cierra el espiche y se empieza uno nuevo. Por defecto son 7 días.

`retention.ms` dice cuánto tiempo se guarda un espiche **ya cerrado** antes de botarlo. Por defecto también son 7 días.

LA FRASE QUE NO ESTÁ EN NINGUNA DOCUMENTACIÓN DE UNA LÍNEA

El segmento activo no se bota nunca. Esté como esté de viejo, mientras sea el que está en uso, se queda. Solo los cerrados son candidatos a irse.

Esa frase explica el 90 % de los «configuré retención de un minuto y no pasó nada» que vas a escuchar en tu carrera. Si `segment.ms` son 7 días y `retention.ms` es 1 minuto, **no se borra nada durante 7 días**. Todos los mensajes, viejos y nuevos, viven en el mismo espiche activo, y ese espiche no se puede botar.

Una decisión que parece de comodidad y no lo es

Un identificador opcional que se le pega a cada mensaje al escribirlo, aparte de su contenido. En SUNAT sería el RUC, o el número de serie del comprobante. **Es lo que Kafka usa para decidir en qué partición cae el mensaje.** Hoy vamos a escribir todo sin clave, y eso tiene una consecuencia.

El tópico va a tener **una sola partición**. Eso es lo que hace que el experimento funcione, y conviene entender por qué antes de teclear.

Cuando los mensajes no llevan clave, el productor de consola **se pega a una sola partición durante toda su sesión**, y en la sesión siguiente elige otra. Con varias particiones, la primera ráfaga cae entera en una y la segunda, que es la que tiene que cerrar el espiche, cae en otra. El espiche que contiene los datos nunca rota, nunca se cierra, y no se borra nada.

Esto está medido sobre este mismo clúster. Con 3 particiones el borrado tardó 265 segundos y dejó una partición sin tocar. Con 1 partición tardó 128 segundos y se llevó todo lo vencido.

El comando

```
kafka-cli/create-topic.sh novatech.lab05.efimero \  
  --partitions 1 \  
  --rf 3 \  
  --config retention.ms=60000 \  
  --config segment.ms=10000
```

kafka-cli/create-topic.sh — un envoltorio del curso. Por dentro ejecuta `kafka-topics --create` dentro del contenedor, y antes te imprime una ficha que explica el comando real

novatech.lab05.efimero — el nombre del tópico

--partitions 1 — **el valor que decide si el laboratorio funciona.** Un solo pedazo, un solo espiche, un solo contador que mirar

--rf 3 — tres copias de esa partición, en tres brokers distintos. El mismo nivel de resguardo que el resto del clúster

--config retention.ms=60000 — **60 segundos.** Un espiche cerrado hace más de un minuto es candidato a irse

--config segment.ms=10000 — **10 segundos.** El espiche se cierra rapidísimo, para que haya candidatos que botar

LO QUE VAS A VER · RECORTADO, LA FICHA DEL ENVOLTORIO ES MÁS LARGA

```
[Create Topic] novatech.lab05.efimero  
Particiones:      1  
Replication factor: 3  
Configs personalizadas:  
  retention.ms=60000  
  segment.ms=10000
```

WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.

Created topic novatech.lab05.efimero.

✓ Tópico novatech.lab05.efimero creado

VAS BIEN SI...

Salió la línea `Created topic novatech.lab05.efimero.` y debajo el ✓ verde del envoltorio.

ESE WARNING NO ES UN ERROR

Sale siempre con este tópico, porque su nombre lleva puntos. Kafka avisa que un nombre que mezcla puntos y guiones bajos podría chocar en sus métricas internas. **El tópico se creó igual**, y la línea `Created topic` que va debajo lo confirma.

SI DICE QUE EL TÓPICO YA EXISTE

El mensaje es `Topic 'novatech.lab05.efimero' already exists` . Te pasa si estás repitiendo el laboratorio. **Bórralo y vuelve a crearlo**, porque necesitas empezar con los contadores en cero:

```
kafka-cli/delete-topic.sh novatech.lab05.efimero
```

Ese script te va a pedir que escribas el nombre del tópico para confirmar. Escríbelo completo y pulsa Enter. Es la protección para que nadie borre un tópico por accidente.

Describirlo y leer la línea Configs

Qué vamos a hacer

Antes de escribirle nada al tópico, hay que saber leerlo. La salida de `describe` es la ficha técnica de un tópico, y es lo primero que vas a mirar el día que algo raro pase en producción.

CONCEPTO · BROKER

Cada uno de los **servidores Kafka** del clúster. Este laboratorio levanta tres, que se llaman `kafka-broker-1`, `kafka-broker-2` y `kafka-broker-3`. Es el mozo.

CONCEPTO · RÉPLICA E ISR

Una **réplica** es cada una de las copias de una partición. Son las libretas de respaldo.

El **ISR** (*In-Sync Replicas*) es cuáles de esas copias están **al día en este momento**. Si el ISR tiene los mismos números que las réplicas, ninguna se quedó atrás. Son los que marcaron tarjeta.

El comando

```
kafka-cli/describe-topic.sh novatech.lab05.efimero | head -2
```

kafka-cli/describe-topic.sh — envoltorio del curso. Por dentro llama a `kafka-topics --describe` dentro del contenedor

novatech.lab05.efimero — qué tópico queremos mirar

| **head -2** — la tubería `|` pasa la salida al siguiente comando. El listado completo trae más de treinta configuraciones. **Las dos primeras líneas son el resumen del tópico y la de su única partición**, que es lo que nos interesa

EL COMANDO REAL, EL QUE VAS A ESCRIBIR EN SUNAT

En el servidor de producción no hay Docker ni envoltorios. El comando es este:

```
kafka-topics --bootstrap-server kafka-broker-1:29092 \
--describe --topic novatech.lab05.efimero
```

- `--bootstrap-server` — la puerta de entrada al clúster. **Le preguntas a un broker cualquiera y él te contesta por todos.** No hace falta que sea el que tiene el dato
- `--describe` — solo consulta. **No modifica nada**, así que se puede ejecutar sin miedo en producción
- `--topic` — sobre qué tópico preguntamos

LO QUE VAS A VER · DOS LÍNEAS MUY LARGAS, QUE AQUÍ SE PARTEN PARA QUE QUEPAN

```
Topic: novatech.lab05.efimero TopicId: JpW30WWSliVh015ODr0QA PartitionCount: 1
ReplicationFactor: 3 Configs: min.insync.replicas=2,retention.ms=60000,segment.ms=10000
Topic: novatech.lab05.efimero Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Elr: LastKnownElr:
```

Cómo se lee la primera línea

Campo	Qué dice
<code>PartitionCount: 1</code>	El tópico tiene un solo pedazo, porque lo pediste así. Es el número que limita cuántos consumidores pueden trabajar en paralelo , y eso lo vas a ver en el laboratorio 06
<code>ReplicationFactor: 3</code>	De ese pedazo hay 3 copias, en 3 brokers distintos. Aguantas perder uno
<code>Configs: ...</code>	Esta es la parte que importa hoy. Trae tres valores. Los dos plazos que acabas de poner, y uno que ya venía del broker

LA TRAMPA CLÁSICA DE LA LÍNEA CONFIGS

`Configs` **no lista la configuración del tópico. Lista solo lo que está cambiado respecto del valor de fábrica.**

Tu tópico tiene más de treinta configuraciones. Aparecen tres porque hay tres cosas cambiadas.

En un tópico que nadie tocó, esta línea se lee casi vacía y se entiende como «este tópico no tiene configuración». Lo que dice en realidad es **«este tópico no tiene ningún valor distinto del de fábrica»**. Los otros treinta y pico están ahí, con su valor por defecto, decidiendo cosas.

Uno de los que **no** aparece es `cleanup.policy`, y es justamente el que decide qué pasa cuando un espiche vence. Está en su valor de fábrica, que es `delete`. El espiche cerrado se bota entero, y eso es lo que vas a ver en los pasos siguientes.

Cómo se lee la segunda línea

Campo	Qué dice
<code>Partition: 0</code>	El número de este pedazo. Solo hay uno, así que solo está el 0
<code>Leader: 1</code>	Qué broker atiende las lecturas y escrituras de este pedazo. Los otros dos solo copian. A ti te va a salir otro número , porque Kafka reparte los líderes al crear
<code>Replicas: 1,2,3</code>	En qué brokers está
<code>Isr: 1,2,3</code>	Cuáles están al día. Los tres
<code>Elr</code> y <code>LastKnownElr</code>	Campos nuevos de Kafka 4.x, para escenarios de pérdida más severos que los de este lab. Aquí salen siempre vacíos

VAS BIEN SI...

`PartitionCount` dice **1**, `ReplicationFactor` dice **3**, y en `Configs` aparecen `retention.ms=60000` y `segment.ms=10000`.

Y en la segunda línea, `Isr` **trae los mismos tres números que** `Replicas`, en el orden que sea. Eso significa que las tres copias están al día.

SI EL ISR TRAE MENOS DE TRES NÚMEROS

Por ejemplo `Replicas: 1,2,3` pero `Isr: 1,2`. Significa que un broker se quedó atrás o se cayó.

Míralo con `docker ps --filter name=kafka-broker`. Si uno no está en `Up (healthy)`, espera un minuto y vuelve a mirar. Si sigue caído, levanta el lab de nuevo con `bin/start-lab.sh`.

Escribir 100 comprobantes y contarlos

Qué vamos a hacer

Vamos a llenar el tópicos y a aprender a contar lo que hay dentro. Para contar mensajes en Kafka no se usa un `SELECT COUNT(*)`. Se usan los dos extremos del registro.

CONCEPTO · OFFSET

El **número de orden de un mensaje dentro de su partición**. Empieza en 0 y nunca se reutiliza. Es el número de comanda, y es correlativo. **Si el comprobante o se botó, el siguiente sigue siendo el 1**. Los números no se corren hacia atrás.

Los dos extremos que importan son estos. `--time -1` te da el offset más nuevo, o sea dónde va a caer el próximo mensaje que se escriba. Es la marca de «hasta aquí escribimos». `--time -2` te da el offset más antiguo que todavía existe. Es la marca de «desde aquí se puede leer».

LA CUENTA QUE VAS A USAR TODA LA CLASE

mensajes vivos = (`--time -1`) menos (`--time -2`)

Cuando Kafka bota un espiche, el offset más nuevo **no se mueve**, porque nadie escribió nada. El que se mueve es el más antiguo, que salta hacia adelante. **Ese salto es el borrado, y es el único rastro que vas a tener.**

Primero, escribe los 100

```
kafka-cli/produce-bulk.sh novatech.lab05.efimero 100
```

kafka-cli/produce-bulk.sh — envoltorio del curso. Fabrica los mensajes solo, con contenido de relleno, y se los pasa a `kafka-console-producer`

novatech.lab05.efimero — a qué tópicos se escribe

100 — cuántos mensajes generar

LO QUE VAS A VER · RECORTADO, SIN LA FICHA DEL ENVOLTORIO

```
[Produce Bulk] 100 mensajes -> novatech.lab05.efimero
```

✓ 100 mensajes publicados en 1s (~100 msg/seg)

ESA CIFRA DE MSG/SEG NO ES UNA MEDICIÓN

La calcula el envoltorio dividiendo mensajes entre segundos, y la mayor parte de esos segundos son el arranque de la máquina virtual de Java. **A ti te va a salir otro número**, y en Docker Desktop sobre Windows suele ser bastante menor. **Sirve para comparar entre corridas, no para decir cuánto rinde Kafka.** Para eso está el laboratorio 07, que usa la herramienta oficial de rendimiento.

Ahora pregunta por los dos extremos

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab05.efimero --time -1

docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab05.efimero --time -2
```

docker exec — ejecuta el comando **dentro** de un contenedor que ya está corriendo. **En el servidor de SUNAT esta parte no existe**, porque el comando se llama solo

kafka-broker-1 — en cuál de los tres contenedores. Cualquiera sirve, porque le pregunta al clúster entero

kafka-get-offsets — la herramienta que pregunta por los extremos del registro. **No lee mensajes, solo cuenta**

--bootstrap-server kafka-broker-1:29092 — el puerto **interno** del broker, porque con `docker exec` estamos hablándole desde dentro de la red de Docker. Desde tu máquina sería `localhost:9092`

--topic — sobre qué tópico preguntamos

--time -1 — el offset más nuevo. Dónde va a caer el próximo mensaje

--time -2 — el offset más antiguo que todavía existe. Desde dónde se puede leer

LO QUE VAS A VER · UNA LÍNEA POR COMANDO

```
novatech.lab05.efimero:0:100
novatech.lab05.efimero:0:0
```

El formato es `tópico:partición:offset`.

Línea	Traducción
<code>novatech.lab05.efimero:0:100</code>	Partición 0, offset más nuevo igual a 100. Se escribieron 100 mensajes, numerados del 0 al 99
<code>novatech.lab05.efimero:0:0</code>	Partición 0, offset más antiguo igual a 0. No se ha botado nada todavía

VAS BIEN SI...

El primer comando termina en `:100` y el segundo en `:0`. **Mensajes vivos: 100 menos 0, o sea 100.**

Anota ese par de números en un papel. Todo el laboratorio se juega en que el segundo deje de ser 0.

SI `PRODUCE-BULK.SH` SE QUEDA COLGADO SIN DECIR NADA

Casi siempre es que **el tópico no existe**. Te saltaste el paso 2, o lo escribiste con una letra distinta.

Este clúster tiene **la creación automática de tópicos desactivada a propósito**, así que el productor no falla. Reintenta en silencio, para siempre.

Corta con `Ctrl+C`, comprueba el nombre con `kafka-cli/list-topics.sh`, crea el tópico y repite.

SI SALE `ERROR: NO SUCH CONTAINER: KAFKA-BROKER-1`

El clúster no está arriba. Vuelve al paso 1 y ejecuta `bin/start-lab.sh`.

Cerrar el espiche

Qué vamos a hacer

Este es el paso que casi todo el mundo se salta, y por saltárselo concluye que la retención no funciona.

LO QUE `SEGMENT.MS` SIGNIFICA DE VERDAD

`segment.ms=10000` **no** quiere decir que un temporizador cierre el espiche a los 10 segundos.

Quiere decir que **la próxima vez que llegue un mensaje**, si el espiche tiene más de 10 segundos de vida, se cierra y se abre uno nuevo.

Kafka no rota espiches en un tópico donde no está pasando nada. Y no lo hace por una razón sensata. Cerrar un espiche cuesta trabajo en disco, y no tiene sentido gastarlo en un tópico que nadie está usando.

La consecuencia es directa. **Si escribes 100 mensajes y te vas a esperar, no se borra nada nunca.** Los 100 quedan en el espiche activo, el espiche activo no se bota, y puedes esperar hasta mañana. Necesitas escribir algo más, después de que pasen los 10 segundos, para que el espiche viejo se cierre.

Espera quince segundos y escribe unos pocos más

```
kafka-cli/produce-bulk.sh novatech.lab05.efimero 5
```

5 — cinco mensajes bastan. **Lo que importa no es cuántos, sino que lleguen después de los 10 segundos** de vida del espiche

OJO CON EL CONTADOR

Con estos 5 mensajes extra, `--time -1` ya no es 100 sino **105**. Los 5 nuevos están en el espiche activo, que es el que no se puede botar.

Ahora mira los archivos que el broker tiene en disco

```
MSYS_NO_PATHCONV=1 docker exec kafka-broker-1 \
  ls /var/lib/kafka/data/novatech.lab05.efimero-0
```

MSYS_NO_PATHCONV=1 — obligatorio en Git Bash sobre Windows. Sin esto, Git Bash convierte `/var/lib/...` en una ruta de tu disco C: antes de que Docker la vea, y el comando falla con un error que no

tiene nada que ver con el problema real. **En macOS y en Linux la variable se ignora**, así que ponerla no molesta

`docker exec kafka-broker-1` — ejecuta dentro del contenedor del primer broker

`ls` — lista los archivos de un directorio

`/var/lib/kafka/data/novatech.lab05.efimero-0` — el directorio donde el broker guarda los espiches de esa partición. **El nombre del directorio es `<tópico>-<partición>`**, así que el `-0` del final es la partición 0

LO QUE VAS A VER

```
000000000000000000000000.index
000000000000000000000000.log
000000000000000000000000.timeindex
000000000000000000000100.index
000000000000000000000100.log
000000000000000000000100.snapshot
000000000000000000000100.timeindex
leader-epoch-checkpoint
partition.metadata
```

Archivo	Qué es
<code>000000000000000000000000.log</code>	El primer espiche. Su nombre es el offset del primer mensaje que contiene , que es el 0. Ya está cerrado, y tiene el reloj corriendo
<code>000000000000000000000100.log</code>	El espiche nuevo, el activo. Arranca en el offset 100, o sea que trae los 5 últimos
<code>.index</code> y <code>.timeindex</code>	Índices, para no tener que recorrer el <code>.log</code> entero cada vez que se busca algo
<code>partition.metadata</code>	La identidad de esta partición. No es un espiche

VAS BIEN SI...

Ves dos archivos `.log`, con nombres distintos. Uno que arranca en ceros y otro que arranca en 100.

Ese primer `.log` es el que tiene los 100 comprobantes, está cerrado, y es el que vas a ver desaparecer.

SI SOLO VES UN `.LOG`

El espiche no rotó, y sin eso el resto del laboratorio no va a mostrar nada, porque no hay ningún espiche cerrado que botar.

Casi siempre es que los 5 mensajes llegaron demasiado pronto, antes de que el espiche cumpliera sus 10 segundos.

Espera diez segundos más, vuelve a escribir 5 mensajes y repite el `ls`. Puedes hacerlo las veces que haga falta.

SI SALE `NO SUCH FILE OR DIRECTORY` CON UNA RUTA RARA QUE EMPIEZA POR `C:/`

Se te olvidó el `MSYS_NO_PATHCONV=1` del principio del comando, y Git Bash convirtió la ruta. Vuelve a ejecutarlo completo.

Esperar, y contar de nuevo

Qué vamos a hacer

Ya están puestas tres de las cuatro condiciones para que el borrado ocurra.

Condición	Cómo la cumpliste
El espiche <code>0</code> está cerrado	Paso 5, con la segunda tanda de mensajes
Sus mensajes tienen más de 60 segundos	Paso 2, con <code>retention.ms=60000</code>
La política es <code>cleanup.policy=delete</code>	Viene de fábrica. Es la que no aparecía en <code>Configs</code>
Que pase la ronda del broker	Esta no depende de ti. Hay que esperar

CONCEPTO · LA RONDA DEL BROKER

`log.retention.check.interval.ms` es cada cuánto el broker se molesta en mirar si hay espiches vencidos. **Por defecto son 5 minutos.** Kafka no borra en el instante en que vence el plazo. Borra en la próxima pasada de la ronda.

Esto no es un defecto del laboratorio. Es exactamente el comportamiento del clúster de producción, y es la mitad de la afirmación que estamos demostrando. **No te avisa, y ni siquiera está mirando todo el tiempo.**

El comando, repetido cada tanto

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab05.efimero --time -2
```

`--time -2` — solo nos interesa el offset más antiguo. **El más nuevo ya sabemos que no se va a mover**, porque nadie está escribiendo

Ejecútalo, espera un minuto, y ejecútalo otra vez. Repite hasta que el número cambie. **Puede tardar hasta cinco minutos.**

PRIMERO, VARIAS VECES, LO MISMO DE ANTES

```
novatech.lab05.efimero:0:0
```

Y EN ALGÚN MOMENTO, SIN QUE NADIE HAYA EJECUTADO NADA

```
novatech.lab05.efimero:0:105
```

La cuenta final

Medición	Antes	Después
offset más nuevo (<code>--time -1</code>)	105	105 · no se movió, nadie escribió
offset más antiguo (<code>--time -2</code>)	0	105
mensajes vivos	105	0

No quedó ninguno. Los 100 comprobantes de la primera ráfaga y los 5 de la segunda, todos. Para cuando llegó la ronda del broker, los 5 de la segunda tanda también habían pasado los 60 segundos, así que también estaban vencidos.

SI A TI TE SALIÓ 100 EN VEZ DE 105

Está igual de bien. Significa que la ronda llegó antes y los 5 últimos todavía no habían vencido. Te quedan 5 vivos en lugar de 0. **Lo que se demuestra es lo mismo.** Se fue todo lo que había pasado el plazo, y nadie ejecutó nada.

LO LOGRASTE SI...

El número que valía 0 dejó de valer 0. Eso es todo. Eso es el borrado.

No hay una alerta, no hay una entrada en un log de auditoría, no hay una métrica que saltara. **El único rastro es ese número.**

SI TRAS CINCO MINUTOS EL NÚMERO SIGUE EN 0

Casi siempre es el paso 5. **Vuelve a mirar si hay dos archivos** `.log` . Si hay uno solo, el espiche nunca se cerró y no hay nada que botar.

```
MSYS_NO_PATHCONV=1 docker exec kafka-broker-1 \  
ls /var/lib/kafka/data/novatech.lab05.efimero-0
```

Escribe 5 mensajes más, comprueba que aparezca el segundo `.log` , y espera la siguiente ronda.

SI EL OFFSET SALTÓ PERO A UN NÚMERO RARO, NI 100 NI 105

Te pasa si repetiste alguna tanda de mensajes, y no es un problema. **Lo que hay que mirar no es el número exacto, sino que dejó de ser 0.** Compara con el `--time -1` de ese momento para saber cuántos te quedan vivos.

Ver el borrado en el disco

Qué vamos a hacer

El contador ya te dijo que los mensajes se fueron. Ahora vamos a mirar el disco, porque ahí se ve **cómo** Kafka los bota, y es en dos tiempos.

El comando

```
MSYS_NO_PATHCONV=1 docker exec kafka-broker-1 \
  ls /var/lib/kafka/data/novatech.lab05.efimero-0
```

LO QUE VAS A VER, SI LLEGAS RÁPIDO

```
000000000000000000000000.index.deleted
000000000000000000000000.log.deleted
000000000000000000000000.timeindex.deleted
000000000000000000000100.index.deleted
000000000000000000000100.log.deleted
000000000000000000000100.snapshot.deleted
000000000000000000000100.timeindex.deleted
000000000000000000000105.log
000000000000000000000105.snapshot
leader-epoch-checkpoint
partition.metadata
```

KAFKA BORRA EN DOS TIEMPOS

Los espiches viejos siguen ahí, pero **renombrados a** `.deleted`.

Primero renombra, y **desde ese instante el archivo ya no existe para ningún consumidor**. Un minuto después lo borra de verdad del disco.

Ese minuto es otro parámetro, `file.delete.delay.ms`, que de fábrica vale 60000.

Espera un minuto más y repite el `ls`

LO QUE QUEDA


```
00000000000000000000105.log  
00000000000000000000105.snapshot  
leader-epoch-checkpoint  
partition.metadata
```

Los `.deleted` desaparecieron. Queda un espiche nuevo, vacío, esperando el próximo mensaje. Y fíjate en su nombre. **Arranca en 105, no en 0.** Los números de offset no se corren hacia atrás nunca.

LO LOGRASTE SI...

Viste los archivos con `.deleted` al final, o directamente el estado final con un solo `.log` cuyo nombre arranca en un número que no es cero.

Con eso cierras el círculo. **Pediste un plazo, se cumplió, y el disco quedó limpio sin que tú ejecutaras un borrado.**

QUÉ SIGNIFICA ESTO PARA SUNAT

El día que un tópico de producción pierda datos que alguien esperaba tener, **la primera línea que hay que ir a mirar es la de `Configs`**, con `describe`. No el log de auditoría.

Y hay algo que no tiene vuelta atrás. **Subir la retención hoy no rescata lo que ya se fue.** Un `--alter` mueve el plazo de los mensajes que siguen vivos. Los que se botaron no vuelven.

Lo que aprendiste

1 `retention.ms` sin `segment.ms` no hace nada.

El plazo se cuenta desde que el espiche se **cierra**, no desde que el mensaje se escribió. Lo viste en el paso 5. Los 100 comprobantes no se movieron hasta que escribiste otros 5. Con los valores de fábrica, que son 7 días y 7 días, un dato puede vivir hasta 14 días en un tópico que en el papel dice 7.

2 El segmento activo es intocable.

Un tópico sin escrituras nuevas no borra nada, por vencido que esté. Si tienes un tópico que dejó de recibir tráfico, sus últimos mensajes se quedan ahí para siempre.

3 El borrado no es un evento observable.

No hay alerta. Si alguien tiene que enterarse de que un dato venció, ese aviso lo construyes tú, fuera de Kafka. La única señal es que el offset más antiguo saltó.

4 La retención es una decisión de negocio.

«¿Cuántos días hay que poder reprocesar un comprobante?» no la contesta el equipo de plataforma. La contesta quien responde por el proceso. El equipo de plataforma solo la escribe en `--config`. Y un tópico sin retención declarada también es una decisión, la de quedarse con los 7 días de fábrica.

Para profundizar

Lo que sigue quedó fuera del recorrido de clase por tiempo, no por dificultad. **Todos estos comandos se ejecutaron contra este mismo clúster antes de publicarlos**, así que corren tal cual están escritos. El análisis queda para ti.

Salvo `novatech.fleet.gps`, que el clúster trae creado, los tópicos que aparecen aquí no existen todavía. Cada bloque incluye su creación.

A · Cambiar la configuración en caliente

Este es el que más rinde de todos, porque es la operación que de verdad vas a hacer todas las semanas. Una regulación cambia y hay que pasar la auditoría de 90 días a un año, sin reiniciar nada.

```
kafka-cli/create-topic.sh novatech.audit.events --partitions 6 --rf 3 \
  --config retention.ms=7776000000 --config compression.type=gzip \
  --config min.insync.replicas=2

kafka-cli/alter-topic-config.sh novatech.audit.events --add retention.ms=31536000000
kafka-cli/describe-topic.sh novatech.audit.events | grep -E "^  retention.ms="
```

--add retention.ms=31536000000 — un año en milisegundos. **El tópico sigue atendiendo tráfico mientras el cambio se aplica**

grep -E "^ retention.ms=" — filtra la línea que nos interesa. **Los dos espacios del principio no son decorativos**, anclan al inicio de línea. Sin ellos, `retention.ms` también coincide dentro de `local.retention.ms`

ANTES Y DESPUÉS, RECORTADO

```
antes      retention.ms=7776000000 ... synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=7776000000}
después    retention.ms=31536000000 ... synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=31536000000}
```

Fíjate en lo que no pasó. `DYNAMIC_TOPIC_CONFIG` ya estaba ahí antes del cambio. No lo puso el `--alter`. Lo puso el `--config` con que se creó el tópico. Todo valor que escribas tú, al crear o al modificar, queda marcado así.

Y ahora deshaz el cambio, que es la parte que sorprende:

```
kafka-cli/alter-topic-config.sh novatech.audit.events --delete retention.ms
kafka-cli/describe-topic.sh novatech.audit.events | grep -E "^  retention.ms="
```

LO QUE SALE

```
retention.ms=6048000000 sensitive=false synonyms={}
```

ESTE ES EL CAMINO QUE HAY QUE SABER RECONOCER

Los 90 días desaparecieron y el tópico volvió a los 7 días de fábrica, con el `synonyms` otra vez vacío. Un `--delete` no pone el valor en cero. Lo devuelve al del broker, que casi nunca es el que querías.

B · Los 7 + 7 días de un tópico recién salido de fábrica

Es la versión aritmética de lo que la doble tanda del paso 5 ya te mostró en vivo. El clúster trae creado `novatech.fleet.gps`, con 6 particiones y ningún plazo propio.

```
kafka-cli/describe-topic.sh novatech.fleet.gps \  
| grep -E "^(retention.ms|segment.ms|cleanup.policy)=\""
```

LO QUE SALE

```
cleanup.policy=delete sensitive=false synonyms={DEFAULT_CONFIG:log.cleanup.policy=delete}  
retention.ms=604800000 sensitive=false synonyms={}\  
segment.ms=604800000 sensitive=false synonyms={}
```

Los dos números son el mismo. 604 800 000 milisegundos, divididos entre 1000, entre 60, entre 60 y entre 24, dan **7 días**. El espiche se cierra cada 7 días, y el plazo de 7 días empieza a contar cuando el espiche se cierra. **Un mensaje puede vivir hasta 14 días en un tópico que en el papel dice «7 días de retención»**. Nadie mintió. El plazo no se cuenta desde donde uno cree.

Y ese `synonyms={}` vacío significa «aquí no escribió nadie, esto es lo que manda el broker».

La pregunta que vale. ¿Cuántos de los tópicos que tu equipo tiene hoy en producción están en este estado, y quién decidió que 7 días era el número?

C · Cuatro tópicos, cuatro personalidades

Cada perfil de dato pide una configuración distinta. Créalos y descríbelos uno por uno.

```
# Telemetría de alta frecuencia: 1 hora de retención, compresión rápida
kafka-cli/create-topic.sh novatech.gps.realtime --partitions 12 --rf 3 \
  --config retention.ms=3600000 --config compression.type=lz4 \
  --config segment.ms=600000

# Estado actual por vehículo: compactación en vez de retención
kafka-cli/create-topic.sh novatech.vehicle.state --partitions 6 --rf 3 \
  --config cleanup.policy=compact --config min.cleanable.dirty.ratio=0.1

# Alertas críticas: máxima durabilidad, retención ilimitada
kafka-cli/create-topic.sh novatech.alerts.critical --partitions 3 --rf 3 \
  --config min.insync.replicas=3 --config retention.ms=-1 \
  --config unclean.leader.election.enable=false
```

retention.ms=-1 — sin plazo. **Se guarda para siempre**, hasta que el disco diga otra cosa

min.insync.replicas=3 — hacen falta las tres copias al día para aceptar una escritura

La pregunta que vale. `min.insync.replicas=3` sobre `--rf 3` significa que si **un solo** broker se cae, el tópico deja de aceptar escrituras. ¿En qué caso eso es lo que quieres?

D · La otra política · compactación

`cleanup.policy` acepta también `compact`, que no mira el reloj. Mira la **clave** de cada mensaje y guarda solo el último valor de cada clave, para siempre. Sirve para estados, o sea «¿cómo está ahora?», en vez de eventos, o sea «¿cuándo pasó?».

```
kafka-cli/produce-bulk.sh novatech.vehicle.state 100 --key-pattern NVT
```

--key-pattern NVT — le pone a cada mensaje una clave `NVT-1`, `NVT-2` y así. **Sin clave no hay compactación posible**, porque no habría por qué agrupar

LA TRAMPA DE ESTE BLOQUE

La compactación es asíncrona, y `min.cleanable.dirty.ratio` decide cuándo vale la pena hacerla. **Inmediatamente después de escribir están todos los mensajes.** Ver el resultado toma bastante más que los 60 segundos del recorrido de hoy, así que no te alarmes si no cambia nada en diez minutos.

E · Particiones · el camino de ida

```
kafka-cli/alter-topic-partitions.sh novatech.gps.realtime 18
kafka-cli/alter-topic-partitions.sh novatech.gps.realtime 6
```

El primero funciona. El segundo falla, y el error dice exactamente por qué:

```
Error while executing topic command : The topic novatech.gps.realtime
currently has 18 partition(s); 6 would not be an increase.
```

Por qué falla. La partición de un mensaje se decide con el *hash* de su clave sobre el número de particiones. Si bajaras de 18 a 6, las claves que hoy caen en la partición 14 pasarían a caer en otra, y sus mensajes viejos se quedarían donde están. El orden por clave, que es lo único que Kafka garantiza, se rompería en silencio. **Kafka prefiere fallar.**

F · Verlo con los ojos

Abre **http://localhost:8090** y ve a *Topics*. Compara lo que la interfaz muestra con lo que devolvió la línea de comandos. La pestaña *Settings* de cada tópico es el equivalente visual de `describe-topic.sh`.

Vale la pena hacerlo, porque en producción vas a discutir con gente que solo mira la interfaz. **Saber qué comando hay detrás de cada pantalla es lo que te deja resolver cuando la interfaz no está.**

G · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre estas actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md`. **Míralas después de intentarlo**, no antes.

Antes de cerrar

DEJA EL TERRENO LIMPIO

Si vas a seguir con el laboratorio 06, **puedes dejar el clúster arriba**. Es el mismo.

Si terminas por hoy:

```
bin/stop-lab.sh
```

Y comprueba con `docker ps --filter name=kafka-broker` que no quede nada corriendo.

Las tres preguntas del laboratorio, para responderte a ti mismo antes de cerrar el archivo. ¿Quién borró los comprobantes? ¿Qué comando de los que ejecutaste hoy borró algo? Y si mañana un tópico de producción pierde datos que alguien esperaba tener, ¿cuál es la primera línea que vas a ir a mirar, y con qué comando?